Online assessment preparation guide

Optiver Software Engineer Internship

EUR – Software Engineer Intern 2026/2027

Prepared for Ibrahim

10 November 2026

Your deadline: **12 November 2026** (plan to finish on **11 November**)

How to read the labels in this guide.

[official] comes only from your own invitation emails from Optiver.

[candidate-reported] comes from forums, blogs and review sites. It is anecdotal, often from other roles or years, and Optiver rotates its formats.

[advice] is my own recommendation, not a fact about the assessment.

This guide is unofficial and not affiliated with Optiver. It contains no real assessment questions. The three mock problems are original practice problems. Prepare as much as you like, but during the real test you must work alone, with no AI tools or outside resources.

Contents

1 Start here: the situation in one page **3**

1.1 The assessment at a glance . . . 3

1.2 How scoring probably works, and what we do not know . . . 4

2 Setup and integrity checklist **4**

2.1 Hardware and environment [official] . . . 4

2.2 Software . . . 4

2.3 Integrity [official] . . . 5

3 What candidates report about the coding question **5**

3.1 Reported themes, and where this guide drills them . . . 5

4 Strategy for the 90 minutes **6**

4.1 The time budget [advice] . . . 6

4.2 Reading checklist for long problems . . . 6

4.3 Parsing and fast input/output . . . 6

4.4 Choosing a language . . . 8

4.5 Debugging routine . . . 8

4.6 Using partial credit . . . 8

5 Pattern library **8**

5.1 Redundant feed merger . . . 8

5.2 Circular buffer . . . 9

5.3 LRU cache . . . 9

5.4 Price-time priority limit order book . . . 10

5.5 Sliding window . . . 11

5.6 Running median with two heaps . . . 11

5.7 Event simulation with a heap . . . 12

5.8 FIFO position and PnL tracker . . . 13

5.9 Interval merging . . . 13

5.10 Python toolbox . . . 14

6 Three mock problems in Optiver style **14**

6.1 Mock A: Redundant market-data feeds (35 min) . . . 14

6.2 Mock B: Limit order book (45 min) . . . 16

6.3 Mock C: Position and PnL tracker with a risk limit (40 min) . . . 18

7 Zap-N games **20**

7.1 What to do about Zap-N 21

8 Two-day plan and day-of checklist **21**

8.1 Day-of checklist 22

A Compact CS-knowledge cheat-sheet (lower priority) **22**

B What was verified, and what was not **23**

C Sources **24**

1 Start here: the situation in one page

The 60-second version

- [official] You must do **one coding question** (about **90 minutes**; C++, Java or Python) and **Zap-N** (one or more short neuro games, 2–15 minutes each). The invitation puts the total at about **1 hour 42 minutes**, and says you may take breaks between the two.
- [official] The coding test runs in the **HackerRank Desktop App**, which needs **Windows or macOS** (Linux is not supported), one monitor, a working webcam and no background apps. It is proctored.
- [official] The deadline is **12 November 2026**. The email gives no time of day or time zone, so [advice] finish on 11 November.
- [official] Each assessment can be completed **once per 8 months**, even across roles or email addresses. There is no retake, so go in prepared.
- [advice] Priority order: (1) fix the setup today, (2) learn the time plan in Section 4, (3) do Mock B and Mock A under the clock, (4) skim the patterns, (5) try the Zap-N practice games, (6) ignore Appendix A unless you have spare time.

1.1 The assessment at a glance

	Your invitation says [official]	**Candidates report** [candidate-reported]
Parts	"Optiver – Campus Software Engineer Test 2026 – EUR": 1 coding question. "Zap-N": 1 or more short neuro assessment games.	Earlier cycles: 2 coding questions plus a roughly 20-minute multiple-choice knowledge quiz plus up to 9 Zap-N games. A 2026 review lists "1 hard question, about 75 minutes", a coding-knowledge quiz and Zap-N.
Coding time	About 90 minutes.	75–90 minutes per problem (QuantVault); 120 minutes for 2 problems in a 2023 write-up.
Languages	C++, Java or Python.	—
Platform	HackerRank Desktop App; Windows or macOS only.	Reviews mention HackerRank. One aggregator says CodeSignal, which conflicts with your invitation, so trust the invitation.
Zap-N	Each game lasts 2–15 minutes.	About 9 games and an hour in older cycles; recent reports of fewer.
Total	About 1 h 42 min.	—
Marking	"Our team will review your results." Partial solutions and your problem-solving approach are valued.	Likely hidden test cases: one reviewer who passed only some cases did not progress.
Retakes	Once per 8 months.	—

Check your eligibility before you invest the hours

[advice] On 8 November Optiver emailed that for the *2027 Start* Software Engineer and Quantitative Research internships "your graduation date does not meet the eligibility requirements". Then on 9 November it sent reminders for the *2026/2027* roles, using the same assessment links as before. The reminders may be automatic leftovers. Log in to the assessment portal

and look at the status of each application, and use the contact details in its FAQ if anything is unclear. Because of the once-per-8-months rule, do not start the test until you are ready.

1.2 How scoring probably works, and what we do not know

[official] Optiver writes: "We encourage you to complete the test even if you're unsure about achieving a perfect score. We value partial solutions and your problem-solving approach. Consider working iteratively by first solving the core problem and then refining your solution with edge cases."

[advice] Read that as a recipe. Get a simple correct core working first, then harden it. A coding platform normally grades by running your program against hidden test cases, so correctness on edge cases is probably what separates a pass from a fail, but this is an inference. We do not know the pass mark, how coding is weighted against Zap-N, or whether anyone reads your code.

2 Setup and integrity checklist

Do this today: the HackerRank Desktop App does not run on Linux

[official] Your invitation says you need "a Windows or macOS device (Linux is not supported)". If your daily machine runs Arch Linux or any other Linux, you need a Windows or macOS laptop for the coding part, or a working dual boot, **sorted today (10 November)**. [advice] Do not count on a virtual machine: proctoring software often detects and blocks them, and I have not verified this for this app. If you cannot get a suitable machine, email Optiver through the contact details in the portal FAQ *before* the deadline.

2.1 Hardware and environment [official]

- ☐ Windows or macOS device, charged and plugged in.
- ☐ **Only one monitor connected.** Unplug any second display.
- ☐ Working webcam, with your **face fully visible the whole time**. Light the room from the front and keep the camera at eye level.
- ☐ Mouse connected (Optiver recommends a laptop or desktop *with a mouse*).
- ☐ Stable internet, preferably wired. **No VPN**, because VPNs or lagging networks may affect your performance.
- ☐ Supported browser installed and updated: Chrome, Edge, Safari or Firefox.
- ☐ Quiet room, door closed, phone silenced and out of reach, water on the desk.

2.2 Software

- ☐ [official] Close **all background applications**. [advice] That includes chat apps, cloud sync, screen recorders and notification pop-ups.
- ☐ [official] Expect to be *prompted to download and install the HackerRank Desktop Application* when you start. [advice] Allow ten minutes for this, and keep your administrator password to hand.
- ☐ [advice] Decide your language now (Section 4.4) and do not switch on the day.
- ☐ [advice] Look in the portal FAQ for a system check. I could not verify that one exists.

2.3 Integrity [official]

- Complete the assessments **without AI tools, online resources, or help from others** unless Optiver says otherwise.
- The assessment is **proctored**, and any unauthorised assistance detected **may result in disqualification**.
- [advice] Whether scrap paper is allowed is not stated in your invitation. Check the portal FAQ before you start, and keep anything you use in view of the camera.

What this means for how you prepare

Everything in this guide is for practising *before* the test. That is fine. During the test, rely on the editor, the problem statement and your own head. Practise in a plain editor with no autocomplete assistant or AI plug-in, so that you do not depend on tools you will not have.

3 What candidates report about the coding question

Your invitation says only "1 coding question". Everything below is [candidate-reported] and comes from a mixture of years, offices and roles. Use it to shape your practice, not as a prediction.

- **Long and wordy, but the core is small.** One 2026 reviewer calls Optiver's style "a very descriptive problem" whose main point "could be boiled down into only a few lines". Another source says the SWE problems are long object-oriented simulations where *corner cases and the final method* are where candidates run out of time.
- **LeetCode-medium-ish, but not pure LeetCode.** Reports range from medium-to-easy algorithmic questions to one lengthy object-oriented design question. A Blind reply says "Some OO questions + LeetCode medium/hard. I had graph problems."
- **Design-flavoured tasks.** InterviewQuery mentions data-structure design (for example a queue with a circular buffer), debugging unfamiliar code, and a small trading-style system.
- **Input parsing can bite.** One 2026 reviewer found the questions easy but left one unanswered because HackerRank's input parsing was awkward.
- **A knowledge quiz appeared in earlier cycles**: about 10–20 multiple-choice questions on OOP, data structures, operating systems, networking, big-O and similar. Your invitation lists none, so it gets only a compact appendix here.

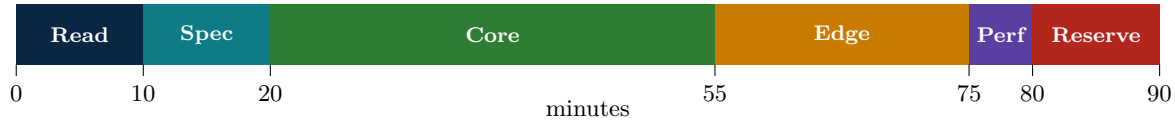
3.1 Reported themes, and where this guide drills them

Reported theme [candidate-reported]	Practise with
Merge two redundant, reordered, duplicated, sequence-numbered packet streams into one in-order, de-duplicated stream (QuantVault)	§5.1, Mock A
Queue built on a circular buffer (InterviewQuery)	§5.2
Order book simulation (a 1point3acres headline for a contractor role; details are behind a login)	§5.4, Mock B
"Allocation backtests" and "trading sequences" (same headline only; contents unknown)	§5.8, Mock C
Hash maps and intervals (a Sydney review)	§5.9, §5.3
Graph problems (a Blind reply)	Appendix A, graph notes
Big-O, caches, concurrency (InterviewQuery)	Appendix A

[advice] The common thread is a stateful simulation: a stream of events or commands, a few data structures that must stay consistent, and precise rules for ties, duplicates and rejections. That is exactly what the patterns in Section 5 and the mocks in Section 6 train.

4 Strategy for the 90 minutes

4.1 The time budget [advice]



Phase	Minutes	What you produce
Read	0–10	Read the statement *twice*: once for the story, once for the rules. Mark every "if", "unless", "at most", "exactly" and tie-break. Copy the sample by hand.
Spec	10–20	Write 6–10 lines of notes: entities, state, each operation's effect, output format, and an edge-case list (use Section 4.2). Choose data structures.
Core	20–55	Write the simplest code that does the main flow. **Run the sample first**, before any edge case.
Edge	55–75	Walk through your edge-case list with tiny hand-made inputs. Fix, re-run the sample after every fix.
Perf	75–80	Check the constraints against your complexity. Replace any $O(N^2)$ scan the limits cannot tolerate.
Reserve	80–90	Final run of all samples, delete debug prints, check output format character by character, submit. No refactoring.

4.2 Reading checklist for long problems

- ☐ What are the **entities** and their **ids**? Can an id be reused? What happens to a duplicate?
- ☐ In what **order** are things processed or matched (time, price, id)? What breaks **ties**?
- ☐ Are boundaries **inclusive or exclusive** ("at least", "more than", "within")? Test the exact boundary value.
- ☐ What happens on an **invalid** operation: ignore, reject with a message, or stop?
- ☐ **Output format**: exact spacing, ordering, case, trailing lines, integers versus decimals, rounding.
- ☐ **Input format**: is a count given first? Blank lines? Strings that contain spaces?
- ☐ **Constraints** → complexity: $N \leq 2 \cdot 10^5$ rules out $O(N^2)$ in Python.
- ☐ Trace the **sample** by hand. Does it exercise every rule? Which rules does it leave untested?
- ☐ List the edge cases: empty input, one element, maximum size, all equal, all duplicates, ties, zero, negatives.

4.3 Parsing and fast input/output

HackerRank problems can be stdin/stdout programs or a function (or class) stub that you complete. [advice] Read what the editor gives you: keep a provided signature exactly, and check whether it wants you to *return* a value or *print* it. Tokens separated by whitespace are safest read with `split()`. If a line can contain spaces, read the line. Strip `\r` if you read lines. Prefer loops to recursion in Python (the default recursion limit is about 1000).

These three templates read a count and then that many integers, and print the sum. Each was

compiled and run on a sample input.

code/io/fast_io.py

```python
import sys


def main():
  data = sys.stdin.read().split()      # every token in one go
  n = int(data[0])
  xs = list(map(int, data[1:1 + n]))
  print(sum(xs))


main()
```

code/io/fast_io.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int n;
  cin >> n;
  long long sum = 0;
  for (int i = 0; i < n; ++i) {
    long long x;
    cin >> x;
    sum += x;
  }
  cout << sum << '\n';
  return 0;
}
```

code/io/Main.java

```java
import java.io.*;
import java.util.*;

public class Main {
  public static void main(String[] args) throws IOException {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    int n = Integer.parseInt(br.readLine().trim());
    StringTokenizer st = new StringTokenizer(br.readLine());
    long sum = 0;
    for (int i = 0; i < n; i++) {
      sum += Long.parseLong(st.nextToken());
    }
    System.out.println(sum);
  }
}
```

4.4 Choosing a language

	Why choose it	Watch out for
Python	Fastest to write and read. `heapq`, `deque`, `OrderedDict`, `Counter`, `bisect` and big integers are built in. Best default for a long simulation problem.	Slow loops at 10^7 operations and beyond. Recursion limit. Mutable default arguments.
C++	Worth it when limits are tight (around 10^6 to 10^7 or more operations) or you are already fluent with the STL.	Verbose. Use `long long` for sums and products. Integer overflow and uninitialised variables are silent.
Java	Reasonable if it is your strongest language.	Verbose class boilerplate. Use `BufferedReader`, never `Scanner`, for large input.

[advice] Pick the language you can write a heap-based simulation in without looking anything up. For most people reading this that is Python. Choose once and rehearse only in that language.

4.5 Debugging routine

1. Run the **sample** and compare the output **exactly** (whitespace and line order too).
2. If it fails, find the **first differing line** and ask which rule of the statement produces that line.
3. **Shrink**: delete input lines until the smallest input that still fails. Trace it by hand.
4. Print state after each operation (temporarily). Compare it with your hand trace.
5. Fix **one thing**, re-run the sample, then re-run your edge cases. Never rewrite wholesale.
6. **Time-box** each bug to about eight minutes. If stuck, note it, move on, and return if time allows.
7. In the last 15 minutes make no structural changes.

4.6 Using partial credit

[official] Optiver values partial solutions. [advice] In practice: (1) get a clearly correct version, even a slow one, working end to end; (2) only then optimise; (3) handle the ordinary cases before the exotic ones; (4) if a hard rule will not come together, make it a conservative default and keep the rest correct; (5) write a short comment where you made an assumption, since a person may read it (this is unconfirmed, but costs nothing). Check at the start of the test how and how often you can submit.

5 Pattern library

Every snippet below lives in `code/patterns/` and is tested by `tests/test_patterns.py`: unit tests plus randomised cross-checks against a deliberately naive brute-force version (200–400 random cases per pattern). I also broke each implementation on purpose to confirm the tests catch it. Type each pattern from memory twice before the test; do not memorise it line by line, understand *why* each line exists.

5.1 Redundant feed merger

Two feeds carry the same numbered packets, possibly late, out of order or duplicated. Release each packet once, in order, as soon as the sequence is contiguous.

code/patterns/stream_merge.py

```python
class StreamMerger:
  """Merge redundant feeds into one in-order, de-duplicated stream."""

  def __init__(self, first_seq=1):
    self.next_seq = first_seq  # next sequence number we may release
    self.pending = {}      # out-of-order packets, keyed by seq

  def push(self, seq, payload):
    """Feed one packet; return the packets released by it, in order."""
    if seq < self.next_seq or seq in self.pending:
      return []        # already released or already buffered
    self.pending[seq] = payload
    out = []
    while self.next_seq in self.pending:
      out.append((self.next_seq, self.pending.pop(self.next_seq)))
      self.next_seq += 1
    return out
```

Pitfalls. Late duplicates after release; duplicates still buffered; the first sequence number may not be 1; a gap that never fills makes the buffer grow without limit (Mock A adds a window); decide which payload wins when duplicates disagree (the first).

5.2 Circular buffer

code/patterns/circular_buffer.py

```python
class CircularBuffer:
  """Fixed-capacity FIFO queue on a plain list; push fails when full."""

  def __init__(self, capacity):
    if capacity <= 0:
      raise ValueError("capacity must be positive")
    self.buf = [None] * capacity
    self.cap = capacity
    self.head = 0        # index of the oldest item
    self.size = 0

  def push(self, x):
    if self.size == self.cap:
      return False
    self.buf[(self.head + self.size) % self.cap] = x
    self.size += 1
    return True

  def pop(self):
    if self.size == 0:
      raise IndexError("pop from empty buffer")
    x = self.buf[self.head]
    self.buf[self.head] = None
    self.head = (self.head + 1) % self.cap
    self.size -= 1
    return x

  def peek(self):
    if self.size == 0:
      raise IndexError("peek at empty buffer")
    return self.buf[self.head]

  def __len__(self):
    return self.size
```

Pitfalls. A full buffer and an empty buffer look alike if you only store head and tail, so keep a `size`. Decide whether a full push *rejects* or *overwrites*, and what capacity zero means.

5.3 LRU cache

code/patterns/lru.py

```python
from collections import OrderedDict


class LRUCache:
  """Least-recently-used cache; get and put are O(1)."""

  def __init__(self, capacity):
    self.cap = capacity
    self.d = OrderedDict()   # oldest first, newest last

  def get(self, key):
    if key not in self.d:
      return -1
    self.d.move_to_end(key)   # mark as most recently used
    return self.d[key]

  def put(self, key, value):
    if key in self.d:
      self.d.move_to_end(key)
    self.d[key] = value
    if len(self.d) > self.cap:
      self.d.popitem(last=False)  # evict the least recently used
```

Pitfalls. `get` must refresh recency. `put` on an existing key updates value *and* recency. Evict only when the size exceeds capacity.

5.4 Price-time priority limit order book

A buy order matches the cheapest sell prices first (earliest arrival first among equals); a sell order matches the highest buy prices. Trades happen at the *resting* order's price. Cancelled orders are not removed from the heap; they are skipped when they reach the top ("lazy deletion").

code/patterns/order_book.py

```python
import heapq


class OrderBook:
  """Price-time priority limit order book with lazy cancellation."""

  def __init__(self):
    self.bids = []   # heap of (-price, seq, oid): best bid on top
    self.asks = []   # heap of (price, seq, oid): best ask on top
    self.live = {}   # oid -> [side, price, remaining quantity]
    self.seen = set()  # every id ever used (ids are never reused)
    self.seq = 0    # arrival counter: breaks price ties by time

  def _best(self, heap):
    while heap and heap[0][2] not in self.live:
      heapq.heappop(heap)       # stale: cancelled or filled
    return heap[0] if heap else None

  def add(self, oid, side, price, qty):
    """Return [(buy_id, sell_id, price, qty), ...], or None if rejected."""
    if oid in self.seen:
      return None
    self.seen.add(oid)
    trades = []
    book = self.asks if side == "B" else self.bids
    while qty > 0:
      top = self._best(book)
      if top is None:
        break
      rid = top[2]
      rprice = top[0] if side == "B" else -top[0]
      if (side == "B" and rprice > price) or (side == "S" and rprice < price):
        break
      rest = self.live[rid]
```

```
      q = min(qty, rest[2])
      buy, sell = (oid, rid) if side == "B" else (rid, oid)
      trades.append((buy, sell, rprice, q))  # trades at the resting price
      qty -= q
      rest[2] -= q
      if rest[2] == 0:
        del self.live[rid]
    if qty > 0:
      self.seq += 1
      self.live[oid] = [side, price, qty]
      if side == "B":
        heapq.heappush(self.bids, (-price, self.seq, oid))
      else:
        heapq.heappush(self.asks, (price, self.seq, oid))
    return trades

  def cancel(self, oid):
    return self.live.pop(oid, None) is not None
```

Pitfalls. The trade price is the resting order's price, not the incoming limit. A partly filled resting order keeps its place in the queue. Remove filled orders from `live`, or a later cancel will wrongly succeed. Never reuse ids. The remainder of an incoming order rests in the book.

5.5 Sliding window

code/patterns/sliding_window.py

```
from collections import deque


def window_max(a, k):
  """Maximum of every length-k window of a, in O(n) with a monotonic deque."""
  dq = deque()    # indices; their values are strictly decreasing
  out = []
  for i, x in enumerate(a):
    while dq and a[dq[-1]] <= x:
      dq.pop()        # smaller values can never be the max again
    dq.append(i)
    if dq[0] <= i - k:
      dq.popleft()      # fell out of the window
    if i >= k - 1:
      out.append(a[dq[0]])
  return out


def window_sums(a, k):
  """Sum of every length-k window of a, in O(n)."""
  out, s = [], 0
  for i, x in enumerate(a):
    s += x
    if i >= k:
      s -= a[i - k]
    if i >= k - 1:
      out.append(s)
  return out
```

Pitfalls. The deque stores *indices*, not values. Use `<=` when popping so equal values do not pile up. A window longer than the input yields an empty result.

5.6 Running median with two heaps

code/patterns/running_median.py

```
import heapq


class RunningMedian:
  """Median of a growing stream: two heaps, O(log n) per insert."""
```

```
6
7   def __init__(self):
8     self.lo = []  # max-heap of the smaller half (stored negated)
9     self.hi = []  # min-heap of the larger half
10
11  def add(self, x):
12    if not self.lo or x <= -self.lo[0]:
13      heapq.heappush(self.lo, -x)
14    else:
15      heapq.heappush(self.hi, x)
16    if len(self.lo) > len(self.hi) + 1:      # rebalance: lo may be
17      heapq.heappush(self.hi, -heapq.heappop(self.lo))  # one larger
18    elif len(self.hi) > len(self.lo):
19      heapq.heappush(self.lo, -heapq.heappop(self.hi))
20
21  def median(self):
22    if not self.lo:
23      raise ValueError("median of empty stream")
24    if len(self.lo) > len(self.hi):
25      return float(-self.lo[0])
26    return (-self.lo[0] + self.hi[0]) / 2
```

Pitfalls. Python has only a min-heap, so negate to get a max-heap. Keep the heaps balanced after *every* insert. For an even count, average the two middle values (check whether the statement wants a float or a rounded value).

5.7 Event simulation with a heap

A queue served by k identical workers. The event heap orders by time, then by event kind, so that a worker finishing at time t is freed *before* a job arriving at t is dispatched.

code/patterns/event_sim.py

```
1  import heapq
2  from collections import deque
3
4
5  def simulate(jobs, k):
6    """FIFO queue served by k identical workers.
7
8    jobs: list of (arrival, duration), arrival non-decreasing, duration >= 1.
9    Returns a list of (start, finish), one per job.
10   """
11   # event = (time, kind, job index); kind 0 = finish sorts before kind 1 = arrival
12   ev = [(a, 1, i) for i, (a, d) in enumerate(jobs)]
13   heapq.heapify(ev)
14   idle, waiting = k, deque()
15   start = [None] * len(jobs)
16   fin = [None] * len(jobs)
17
18   def dispatch(t):
19     nonlocal idle
20     while idle and waiting:
21       i = waiting.popleft()
22       idle -= 1
23       start[i], fin[i] = t, t + jobs[i][1]
24       heapq.heappush(ev, (fin[i], 0, i))
25
26   while ev:
27     t, kind, i = heapq.heappop(ev)
28     if kind == 0:
29       idle += 1
30     else:
31       waiting.append(i)
32     dispatch(t)
33   return list(zip(start, fin))
```

Pitfalls. Always put an explicit tie-breaker in the heap tuple; otherwise Python may try to

compare unorderable payloads. Decide what happens when events share a timestamp, and test it.

5.8 FIFO position and PnL tracker

Buys create long lots and sells create short lots. A trade first closes the oldest lots of the opposite sign (realising profit or loss) and any remainder opens a new lot, so one trade can flip you through zero.

code/patterns/pnl_fifo.py

```python
from collections import deque


class FifoPnL:
  """One symbol, FIFO lots. qty > 0 buys, qty < 0 sells. Integer prices."""

  def __init__(self):
    self.lots = deque()   # [price, signed qty]; all lots share one sign
    self.realised = 0

  @property
  def position(self):
    return sum(q for _, q in self.lots)

  def trade(self, price, qty):
    # first close the oldest opposite-signed lots
    while qty != 0 and self.lots and (self.lots[0][1] > 0) != (qty > 0):
      lot = self.lots[0]
      sign = 1 if lot[1] > 0 else -1   # +1: long lot, -1: short lot
      closed = min(abs(qty), abs(lot[1]))
      self.realised += sign * (price - lot[0]) * closed
      lot[1] -= sign * closed
      qty += sign * closed         # |qty| shrinks toward zero
      if lot[1] == 0:
        self.lots.popleft()
    if qty != 0:                # whatever is left opens a lot
      self.lots.append([price, qty])

  def unrealised(self, mark):
    return sum((mark - p) * q for p, q in self.lots)
```

Pitfalls. A short lot profits when the price *falls*, so the sign flips. Unrealised PnL is $\sum$(mark − lot price) × signed quantity. Keep prices as integers to avoid floating-point noise. Realised PnL survives when the position returns to flat.

5.9 Interval merging

code/patterns/intervals.py

```python
def merge_intervals(iv):
  """Merge closed intervals (a, b) with a < b; touching ones merge too."""
  out = []
  for a, b in sorted(iv):
    if out and a <= out[-1][1]:
      out[-1][1] = max(out[-1][1], b)
    else:
      out.append([a, b])
  return [tuple(x) for x in out]
```

Pitfalls. Sort by start. Decide whether touching intervals (`[1,2]` and `[2,3]`) merge; here they do, and `[1,2]` with `[3,4]` do not. If intervals are half-open or integer-inclusive, the comparison changes.

5.10 Python toolbox

`heapq`	`heappush`, `heappop`, `heapify`; min-heap; tuples compare element by element.
`collections.deque`	$O(1)$ at both ends. Never use `list.pop(0)` in a loop.
`OrderedDict`	`move_to_end`, `popitem(last=False)`.
`defaultdict`, `Counter`	Grouping and counting.
`bisect`	`bisect_left`/`bisect_right` on a sorted list.
`sorted(key=...)`	Stable sort; sort by a tuple for multi-key order.
`sys.stdin.read().split()`	Fast whole-input tokenising.

6 Three mock problems in Optiver style

These are **original** problems written for practice; they are not Optiver questions. Each is a long-story, stateful simulation with a few precise rules, in the style candidates describe. Set a timer for 35–45 minutes, solve one without looking at the reference solution, then mark yourself with the rubric.

Every reference solution was run on the printed sample (whose output I traced by hand first) and cross-checked against an independent brute-force implementation on 600 random inputs per problem. They are in `code/mocks/`, with brute-force versions and tests in `tests/test_mocks.py`.

6.1 Mock A: Redundant market-data feeds (35 min)

Story. A trading system receives market data on two redundant lines, `A` and `B`. Each packet has a sequence number starting at 1 and a payload. Both lines carry the same packets, but packets can arrive late, out of order, more than once, or never. The system must hand packets onwards *exactly once and in order*, but it can only buffer packets whose sequence number is close to the next expected one, to bound memory.

Input. First line: W N. The next N lines, in arrival order: `F S P` where `F` is `A` or `B`, `S` is the sequence number, `P` is a payload with no spaces.

Rules. Let `next` be the smallest sequence number not yet released (initially 1). For each arriving packet, apply the first rule that matches:

1. $S <$ `next`: it was already released. Count it as a *duplicate*.
2. $S \geq$ `next` $+ W$: outside the window. *Drop* it, do not buffer it, and count it as dropped (it may legitimately arrive again later).
3. S is already buffered: count a duplicate. If its payload differs from the buffered payload, also count a *conflict*. Keep the first payload.
4. Otherwise buffer it. Then, while `next` is buffered, release it (print an `OUT` line) and increase `next`.

Output. For each released packet, in order: `OUT S P F`, where `F` is the line that delivered the buffered copy. At the end: `SUMMARY out=`k `dup=`d `dropped=`x `conflict=`c `buffered=`b, where b is the number of packets still buffered.

Constraints. $1 \leq W \leq 10^5$, $0 \leq N \leq 2 \cdot 10^5$, $1 \leq S \leq 10^9$, payload at most 20 characters.

Sample input (left) and output (right).

```
3 11
A 1 p1
B 1 p1
B 3 p3
A 3 p3
A 6 p6
B 3 X3
A 2 p2
B 6 p6
A 5 p5
B 4 p4
A 9 p9
```

```
OUT 1 p1 A
OUT 2 p2 A
OUT 3 p3 B
OUT 4 p4 B
OUT 5 p5 A
OUT 6 p6 B
SUMMARY out=6 dup=3 dropped=1 conflict
    =1 buffered=1
```

Hidden edge cases a real test would include

- $N = 0$; $W = 1$; the first packet is not number 1 and number 1 never comes.
- $S = \texttt{next} + W - 1$ (accepted) versus $S = \texttt{next} + W$ (dropped).
- A duplicate of a buffered packet with a different payload (conflict).
- A late duplicate of a released packet with a different payload (*only* a duplicate).
- A packet dropped now that arrives again after the window has moved.
- One arrival that releases a long run of buffered packets.
- Sequence numbers up to 10^9 (use a dictionary, not an array); trailing blank lines.

Reference solution.

code/mocks/mock_a_feeds.py

```python
import sys


def solve(lines):
  W, n = map(int, lines[0].split())
  nxt = 1            # next sequence number to release
  pending = {}         # seq -> (payload, feed) buffered out of order
  out = []
  released = dup = dropped = conflict = 0
  for line in lines[1:1 + n]:
    feed, s, payload = line.split()
    s = int(s)
    if s < nxt:                # already released
      dup += 1
    elif s >= nxt + W:            # beyond the window: discard
      dropped += 1
    elif s in pending:            # already buffered
      dup += 1
      if pending[s][0] != payload:
        conflict += 1          # keep the first payload seen
    else:
      pending[s] = (payload, feed)
      while nxt in pending:         # release the contiguous run
        p, f = pending.pop(nxt)
        out.append(f"OUT {nxt} {p} {f}")
        released += 1
        nxt += 1
  out.append(f"SUMMARY out={released} dup={dup} dropped={dropped} "
        f"conflict={conflict} buffered={len(pending)}")
  return out


if __name__ == "__main__":
  print("\n".join(solve(sys.stdin.read().split("\n"))))
```

Self-marking rubric (20).

Sample output exactly right	4
Duplicates and late duplicates counted correctly	4
Window boundary exact (`next` $+ W$)	4
Conflict counted only for buffered duplicates, first payload kept	3
A single arrival releases the whole contiguous run	3
Linear time, dictionary buffer	2

Common pitfalls. Measuring the window from the highest sequence seen instead of `next`. Using `list.pop(0)` or `in` on a list. Releasing packets only at the end. Counting a conflict for a late duplicate.

6.2 Mock B: Limit order book (45 min)

Story. You are writing the matching engine of a small exchange for one instrument. Traders send limit orders to buy (`B`) or sell (`S`) at a price for a quantity, and can cancel their resting orders. Orders match by *price-time priority*.

Input. First line: N. Then N commands, either `ADD id side price qty` or `CANCEL id`.

Rules.

1. An `ADD` whose `id` was used by any earlier `ADD` (even if that order was filled or cancelled) is rejected: print `REJECT id` and change nothing.
2. An incoming buy matches resting sells priced at most its limit price; an incoming sell matches resting buys priced at least its limit price. Best price first (lowest sell, highest buy); among equal prices the earlier-arrived order first.
3. Each trade executes at the *resting* order's price for the smaller of the two remaining quantities. Print `TRADE buy_id sell_id price qty`. A partly filled resting order keeps its priority.
4. Any unfilled remainder of the incoming order rests in the book.
5. `CANCEL id` prints `CANCELLED id` if the order is still resting with quantity left, otherwise `REJECT id`.
6. After all commands print `BOOK`, then buy levels from highest price down as `B price total_qty`, then sell levels from lowest price up as `S price total_qty`.

Constraints. $0 \le N \le 2 \cdot 10^5$; ids, prices and quantities in $[1, 10^9]$.

Sample input (left) and output (right).

```
13
ADD 1 B 100 10
ADD 2 B 101 5
ADD 3 S 102 4
ADD 4 S 100 12
CANCEL 3
CANCEL 3
ADD 2 S 99 1
ADD 5 B 103 20
ADD 6 S 100 8
ADD 7 S 100 4
CANCEL 5
ADD 8 B 100 2
ADD 9 S 100 4
```

```
TRADE 2 4 101 5
TRADE 1 4 100 7
CANCELLED 3
REJECT 3
REJECT 2
TRADE 5 6 103 8
TRADE 5 7 103 4
CANCELLED 5
TRADE 1 9 100 3
TRADE 8 9 100 1
BOOK
B 100 1
```

Hidden edge cases a real test would include

- A buy priced exactly at the best sell price (equal prices trade).
- The best resting order was cancelled earlier, so the next one must be hit.
- A big incoming order sweeping several price levels and several orders at one level.
- A partly filled resting order, then a later order that must hit it before younger orders at the same price.
- Cancelling a filled, an already cancelled, or an unknown order; reusing the id of a cancelled order.
- Several resting orders at one price must be summed in the final `BOOK`; an empty book prints only `BOOK`.

Reference solution.

code/mocks/mock_b_book.py

```python
import heapq
import sys


def solve(lines):
  n = int(lines[0])
  bids, asks = [], []      # heaps: (-price, seq, id) and (price, seq, id)
  live = {}           # id -> [side, price, remaining]
  seen = set()
  seq = 0
  out = []

  def best(heap):
    while heap and heap[0][2] not in live:
      heapq.heappop(heap)        # lazily drop cancelled/filled
    return heap[0] if heap else None

  for line in lines[1:1 + n]:
    t = line.split()
    if t[0] == "CANCEL":
      oid = int(t[1])
      if oid in live:
        del live[oid]
        out.append(f"CANCELLED {oid}")
      else:
        out.append(f"REJECT {oid}")
      continue
    oid, side, price, qty = int(t[1]), t[2], int(t[3]), int(t[4])
    if oid in seen:
      out.append(f"REJECT {oid}")
      continue
    seen.add(oid)
    book = asks if side == "B" else bids
    while qty > 0:
      top = best(book)
      if top is None:
        break
      rid = top[2]
      rp = top[0] if side == "B" else -top[0]
      if (side == "B" and rp > price) or (side == "S" and rp < price):
        break
      q = min(qty, live[rid][2])
      buy, sell = (oid, rid) if side == "B" else (rid, oid)
      out.append(f"TRADE {buy} {sell} {rp} {q}")
      qty -= q
      live[rid][2] -= q
      if live[rid][2] == 0:
        del live[rid]
    if qty > 0:
      seq += 1
      live[oid] = [side, price, qty]
```

```
        if side == "B":
          heapq.heappush(bids, (-price, seq, oid))
        else:
          heapq.heappush(asks, (price, seq, oid))

    out.append("BOOK")
    for side, rev in (("B", True), ("S", False)):
      levels = {}
      for s, p, q in live.values():
        if s == side:
          levels[p] = levels.get(p, 0) + q
      for p in sorted(levels, reverse=rev):
        out.append(f"{side} {p} {levels[p]}")
    return out


if __name__ == "__main__":
    print("\n".join(solve(sys.stdin.read().split("\n"))))
```

Self-marking rubric (20).

Sample output exactly right	4
Price priority, then time priority, on both sides	4
Trade at resting price; partial fills handled; remainder rests	4
Cancel and reject rules (filled, cancelled, unknown, reused id)	4
Final book aggregated and sorted correctly	2
Heaps with lazy deletion, not repeated scans	2

Common pitfalls. Sign error for the sell heap. Trading at the incoming limit price. Forgetting to mark filled orders as no longer live. Giving a partly filled order a new timestamp. $O(N^2)$ scanning.

6.3 Mock C: Position and PnL tracker with a risk limit (40 min)

Story. A risk desk tracks several instruments. Each accepted trade changes the position; profit is realised on a first-in-first-out basis, and a per-instrument position limit blocks trades that would breach it.

Input. First line: L N (position limit and number of commands). Then N commands: `TRADE sym side price qty` (`side` is `B` or `S`), `MARK sym price`, or `REPORT`. Prices and quantities are positive integers.

Rules.

1. The position of a symbol is bought minus sold quantity. A `TRADE` is *rejected* (print `REJECT`, change nothing) if the absolute position after it would exceed L. Otherwise print `OK`.
2. An accepted trade first closes the oldest open lots of the opposite direction (FIFO). Closing a long lot realises (sell price − lot price) × q; closing a short lot realises (lot price − buy price) × q. Any quantity left opens a new lot, so one trade may flip a position through zero.
3. The *mark* of a symbol is the price of its latest accepted trade or `MARK`, whichever came last. A `MARK` for a symbol with no accepted trade is ignored. Rejected trades do not change the mark.
4. Unrealised PnL is $\sum$(mark − lot price) × signed lot quantity over open lots (short lots have negative quantity).
5. `REPORT` prints `REPORT`, then for every symbol with at least one accepted trade, in alphabetical order, `sym position realised unrealised`, then `TOTAL realised unrealised`.

Constraints. $1 \le L \le 10^9$; $N \le 2 \cdot 10^5$; at most 1000 symbols; prices up to 10^6. Use 64-bit integers.

Sample input (left) and output (right).

```
5 12
TRADE AAA B 100 3
TRADE AAA B 102 2
TRADE AAA B 103 1
TRADE AAA S 105 4
MARK AAA 104
REPORT
TRADE AAA S 101 4
TRADE BBB S 50 6
TRADE BBB S 50 5
MARK BBB 48
TRADE BBB B 49 2
REPORT
```

```
OK
OK
REJECT
OK
REPORT
AAA 1 18 2
TOTAL 18 2
OK
REJECT
OK
OK
REPORT
AAA -3 17 0
BBB -3 2 3
TOTAL 19 3
```

Hidden edge cases a real test would include

- A trade that takes the position exactly to the limit (accepted) or one past it (rejected).
- A single trade that closes the whole position and flips it (for example long 3, then sell 6).
- A rejected trade followed by a report: the mark must not have moved.
- `MARK` before any trade, or for an unknown symbol.
- A symbol that returns to flat: it still appears with position 0 and keeps its realised PnL.
- Several lots at different prices closed by one trade; `REPORT` with no symbols prints `REPORT` then `TOTAL 0 0`.
- Negative realised or unrealised values (print the minus sign, no rounding).

Reference solution.

code/mocks/mock_c_pnl.py

```python
import sys
from collections import deque


def solve(lines):
  limit, n = map(int, lines[0].split())
  lots = {}     # symbol -> deque of [price, signed qty], one sign only
  real = {}     # symbol -> realised PnL
  mark = {}     # symbol -> latest MARK or accepted trade price
  out = []

  def pos(sym):
    return sum(q for _, q in lots[sym])

  for line in lines[1:1 + n]:
    t = line.split()
    if t[0] == "TRADE":
      sym, side, price, qty = t[1], t[2], int(t[3]), int(t[4])
      signed = qty if side == "B" else -qty
      cur = pos(sym) if sym in lots else 0
      if abs(cur + signed) > limit:       # |new position| over limit
        out.append("REJECT")
        continue
      if sym not in lots:
        lots[sym], real[sym] = deque(), 0
      q = signed
      while q != 0 and lots[sym] and (lots[sym][0][1] > 0) != (q > 0):
        lot = lots[sym][0]
        sg = 1 if lot[1] > 0 else -1
        closed = min(abs(q), abs(lot[1]))
        real[sym] += sg * (price - lot[0]) * closed
```

```
        lot[1] -= sg * closed
        q += sg * closed
        if lot[1] == 0:
          lots[sym].popleft()
      if q != 0:
        lots[sym].append([price, q])
      mark[sym] = price
      out.append("OK")
    elif t[0] == "MARK":
      if t[1] in lots:          # ignore unknown symbols
        mark[t[1]] = int(t[2])
    else:                       # REPORT
      out.append("REPORT")
      tr = tu = 0
      for sym in sorted(lots):
        un = sum((mark[sym] - p) * q for p, q in lots[sym])
        out.append(f"{sym} {pos(sym)} {real[sym]} {un}")
        tr += real[sym]
        tu += un
      out.append(f"TOTAL {tr} {tu}")
  return out


if __name__ == "__main__":
  print("\n".join(solve(sys.stdin.read().split("\n"))))
```

Self-marking rubric (20).

Sample output exactly right	4
FIFO realised PnL across several lots	4
Flip through zero in one trade	4
Limit rule on absolute position after the trade	3
Mark rules (latest MARK or accepted trade, rejects ignored)	3
Output order and formatting	2

Common pitfalls. Using average cost instead of FIFO. Getting the sign wrong for short lots. Updating the mark on rejected trades. Resetting realised PnL when flat. Summing positions inside a loop for 200 000 commands (the reference recomputes position per trade, which is fine for few lots; keep a running position if lots can pile up).

7 Zap-N games

[official] Your invitation says Zap-N "consists of 1 or more short neuro assessment games. Each game lasts between 2 and 15 minutes." Nothing more is stated, including which games you will get or how they are scored. Everything in the table is [candidate-reported], mostly from QuantVault, JobTestPrep and TraderMath. Names and descriptions vary between sources and cycles; I flag what I could not confirm.

Game	What happens (reported)	Tests	Practical tip [advice]
Skyscraper / Tower	Block stacking toward a target arrangement.	Spatial planning	Plan backwards from the goal before the first move.
Code Compare	Match a 7–10 character string against five choices; the decision window shrinks towards about half a second.	Speed, short-term recall, dual-task load	Compare length and the first and last characters first; keep the pace and accept some errors.
The Switch / Task Switch	Circles and squares flash alternately (right arrow for one, left arrow for the other), later mixed with an odd/even arithmetic rule that flips.	Task-switching	Rest your fingers on the arrow keys and do not look at the keyboard.
Balloon	Pump a balloon for points; it may pop. Reported to punish greed once you know the pop distribution.	Risk appetite	Decide a stopping rule in the tutorial and keep to it; do not chase losses.
Pincode	Remember and re-enter digit sequences, getting longer level by level.	Working memory	Chunk digits in pairs and do not let yourself be interrupted.
Number Box / 24-style	Four numbers and a target; combine them with basic arithmetic.	Mental arithmetic	Look for factor pairs of the target; try multiplication first.
Grill Master	Pieces of meat on a grill, each with a progress bar.	Multitasking, prioritising	Always service the item nearest to done or burnt.
Dial-timing reaction	A timing or reaction game (one source's wording; I could not verify details).	Reaction time	Read the tutorial carefully; use the mouse.
Pattern deduction / "Figure It Out"	Deduce a rule from examples. I could not confirm these two names are the same game.	Pattern recognition	Form a hypothesis fast and test it on the next example.
Shapeshift	Listed by two sources; I could not verify what it involves.	—	Read the tutorial.

7.1 What to do about Zap-N

- [candidate-reported] Each game has a **tutorial or trial phase**. Use it to work out the rule and whether time matters.
- [candidate-reported] The system asks which controls you use (touchscreen, trackpad, mouse) and takes that into account in scoring. [advice] Use a mouse, as Optiver recommends.
- [advice] Take it rested and alert. Zap-N rewards calm, not strategy, and you cannot cram for it.
- [advice] Do not over-prepare: 20–30 minutes on free practice versions is enough to remove surprise. Free practice: `https://quantvault.org/zap-n.html` and TraderMath (`https://tradermath.org`).
- [candidate-reported] QuantVault says one weak game section reportedly no longer rejects a candidate on its own, but I could not verify this, so do not rely on it.
- [advice] Your invitation says "1 or more" games. Reports of 9 games come from older graduate cycles; do not be thrown if you get only a few.

8 Two-day plan and day-of checklist

[advice] Your deadline is 12 November, but treat 11 November as the real deadline so that a technical problem still leaves a day. If your Quantitative Research assessment is due the same day

(it uses the same deadline), plan the two in **separate sittings**; you are allowed breaks within each, not a reason to stack them.

When	Hours	Do
Tue 10 Nov, now	0.5	Setup and eligibility check (Sections 1–2): OS, one monitor, webcam, portal status. Fix the Linux problem first if it applies.
Tue 10 Nov, evening	1	Read Sections 3–4. Type the I/O template for your language three times from memory.
	1.5	Patterns §5.1, §5.4, §5.8, §5.7: read, then write each from memory and compare.
	0.5	Three short Zap-N practice rounds.
	—	**Sleep at least 7.5 hours.**
Wed 11 Nov, morning	0.8	Mock A, timed 35 minutes, then mark and review.
	1.0	Mock B, timed 45 minutes, then mark and review.
	1.5	Break: meal, walk. No new material.
	0.8	Mock C, timed 40 minutes, then review.
Wed 11 Nov, afternoon	0.3	Final setup run: close apps, camera, charger, water.
	1.7	**Take the real assessment**, ideally starting by mid-afternoon and well before you are tired.
Thu 12 Nov	—	Spare day for technical trouble only. Do not use the spare day to start.

[advice] If time runs short, drop things from the bottom of this order: Zap-N practice, patterns, Mock C, Mock A. Do not drop the setup check or Mock B.

8.1 Day-of checklist

- ☐ Slept; ate; water on the desk; phone silenced and out of reach.
- ☐ Windows or macOS machine plugged in; **one monitor**; mouse connected.
- ☐ Webcam on, face fully visible, front lighting; door closed.
- ☐ No VPN; background apps closed; notifications off.
- ☐ Desktop app installed when prompted (allow ten minutes).
- ☐ Language chosen; I/O template in your head; time plan visible on your mind: 10 read, 10 spec, 35 core, 20 edge, 5 perf, 10 reserve.
- ☐ Open the problem and **read it twice before typing**.
- ☐ Run the sample before anything else; keep a list of edge cases and tick them off.
- ☐ Final five minutes: rerun the sample, remove debug output, check the output format, submit.
- ☐ Afterwards: do not discuss the questions publicly, and note anything unusual to tell Optiver if there was a technical fault.

A Compact CS-knowledge cheat-sheet (lower priority)

[candidate-reported] Earlier cycles added a quiz of roughly 10–20 multiple-choice questions on OOP, data structures, operating systems and networking, and some reviews mention big-O, cache locality, garbage collection and concurrency. [official] Your invitation lists no quiz. Skim this only after everything else.

Topic	One-line answers
Complexity	List append $O(1)$ amortised; dict/set lookup $O(1)$ average; heap push/pop $O(\log n)$; sort $O(n \log n)$; binary search $O(\log n)$; BFS/DFS $O(V + E)$; Dijkstra with a heap $O((V + E) \log V)$.
Data structures	Array: fast index, slow middle insert. Linked list: fast insert, slow index. Hash table: average $O(1)$, worst $O(n)$ with collisions. Heap: priority queue. Balanced tree: ordered, $O(\log n)$.
OOP	Encapsulation hides state; inheritance reuses by "is-a" (prefer composition for "has-a"); polymorphism lets one interface have many implementations; an interface specifies behaviour without data.
OS	Process: own address space. Thread: shares memory with its process. Context switch: save and restore CPU state. Mutex: one holder; semaphore: counted permits. Deadlock needs mutual exclusion, hold-and-wait, no pre-emption and circular wait. Virtual memory maps pages to frames; a page fault loads a page.
Caches and memory	Contiguous access is fast (spatial locality): traverse row by row in a row-major array. Stack: automatic, per-call; heap: dynamic allocation. Garbage collection reclaims unreachable objects and may pause the program.
Networking	TCP: reliable, ordered, connection set up by a three-way handshake, with flow and congestion control. UDP: connectionless, no delivery guarantee, lower latency. Latency is delay; throughput is volume per second.
Concurrency	Race condition: result depends on timing. Atomic operations and locks protect shared data; lock-free structures avoid locks but are hard to get right. Optimistic locking checks for conflicts at commit; pessimistic locking locks first.
Graph notes	BFS finds shortest paths in unweighted graphs; DFS explores depth first and suits cycle detection and components; Dijkstra needs non-negative weights; topological order exists only for DAGs.

B What was verified, and what was not

Verified by running. Every code listing in this guide is read directly from files that passed the tests. `tests/test_patterns.py` (19 tests) and `tests/test_mocks.py` (13 tests) run unit tests, hand-verified sample outputs, and randomised cross-checks against separate brute-force implementations (400 or 600 random cases each). I also introduced ten deliberate bugs (for example inverted tie-breaks, off-by-one at the window boundary); the tests caught nine, and the tenth was harmless (equal values may go to either heap). The three I/O templates were compiled and run (Python 3, g++ 17, Java). Run everything with `run_all.sh`.

Not verified.

- The real test's problem type, scoring, pass mark, and whether code is read by a person.
- Whether a system check, a virtual machine, or scrap paper is allowed or possible with the HackerRank Desktop App.
- The deadline's time of day and time zone.
- The contents of the 1point3acres threads (login wall; only the headline was visible).
- The exact Zap-N game set you will get, and several game descriptions (marked in the table).
- All **[candidate-reported]** claims are anecdotes from other years, offices and roles.

C Sources

All are third-party and unaffiliated with Optiver, and are anecdotal. Your invitation emails from Optiver are the only **[official]** source.

- QuantVault, *Optiver Online Assessment*: https://quantvault.org/optiver-online-assessment.html
- QuantVault, *Zap-N games*: https://quantvault.org/zap-n.html
- vijaySadhuram, *Optiver recruitment, Grad SWE 2023* (GitHub): https://github.com/vijaySadhuram/Optiver-Recruitment-Grad-SWE
- ProgramHelp, *Optiver 2026 New Grad OA guide*: https://programhelp.net/en/optiver-oa/
- InterviewQuery, *Optiver software engineer interview guide*: https://www.interviewquery.com/guides/optiver-software-engineer
- Glassdoor, a 2026 review ("very descriptive problem"): https://www.glassdoor.sg/Interview/Optiver-Interview-E243355-RVW23293347.htm
- Glassdoor, a 2026 review (input parsing): https://www.glassdoor.sg/Interview/Optiver-Interview-E24htm
- Glassdoor, a February 2026 graduate review (1 hard question, quiz, Zap-N): https://www.glassdoor.fr/Entretien/Optiver-Entretien-E243355-RVW80446991.htm
- Blind, *Optiver Amsterdam SWE HackerRank OA*: https://www.teamblind.com/post/optiver-amsterdam-s
- 1point3acres, programming challenges thread (headline only): https://www.1point3acres.com/interview/thread/1154944
- TraderMath, *Mastering the Optiver Zap-N test*: https://tradermath.org/knowledge-base/mastering-the-optiver-zap-n-test
- JobTestPrep, *Optiver test* (Zap-N descriptions): https://www.jobtestprep.co.uk/optiver-test
- DEV Community, *Optiver 2026 OA comprehensive review*: https://dev.to/net_programhelp_e160eef28/optiver-2026-oa-comprehensive-review-26ng-intern-full-guide-eld
- DEV Community, *Optiver OA experience, 90 min coding*: https://dev.to/interview-aid/optiver-oa-experience-90min-coding-probability-logic-game-test-breakdown-5ek8